TSD 2026 — Software Testing Project

Team Crousty

Application: Demo Web Shop · `http://localhost:3001`

Submission: 9 July 2026

1. Application Overview

Demo Web Shop — a single-page e-commerce application

Detail	Value
Stack	Node.js · json-server 0.17.4 · Vanilla JS
Routing	Hash-based SPA (`/#/login` , `/#/cart` , …)
URL	`http://localhost:3001` (local)

Main features tested

- **User Authentication** — login / error handling
- **Product Search** — keyword search, results display
- **Shopping Cart** — add to cart, quantity, notifications
- **Checkout Process** — guest and registered flows
- **Boundary validation** — qty limits, password length

2. Test Strategy

```
          10 Manual Test Cases (TC-001 → TC-010)
                ↓              ↓              ↓
      Selenium (3)     Robot FW (2)    Postman (API)
      TC-001/002/003   TC-001/004      REST endpoints
```

Tool	Language	TC covered
JUnit 5	Java	Rating unit tests (Lab 2)
Selenium WebDriver 4.18.1	Java	TC-001, TC-002, TC-003
Robot Framework 7.1.1	Python	TC-001, TC-004
Postman	—	REST API layer

Automation criteria: stable UI flow · clear assertion · high regression value

3. Manual Test Cases

TC ID	Title	Type	Status
TC-001	Successful login	Positive	✅ PASS
TC-002	Search existing product	Positive	✅ PASS
TC-003	Add item to cart	Positive	✅ PASS
TC-004	Login with wrong password	Negative	✅ PASS
TC-005	Search non-existing product	Negative	✅ PASS
TC-006	Checkout with empty cart	Negative	✅ PASS
TC-007	Password minimum length	Boundary	✅ PASS
TC-008	Add zero/negative quantity	Boundary	❌ BUG-001
TC-009	Full checkout – Guest	Flow	✅ PASS
TC-010	Full checkout – Registered	Flow	✅ PASS

8/10 PASS · 2 defective (BUG-001)

4. Defect Report — BUG-001

Title: System accepts quantity 0 silently — no validation error

Field	Value
Severity	Medium
Component	Shopping Cart
Steps	Open product page → enter qty `0` → click Add to cart
Expected	Error: "Quantity must be greater than 0"
Actual	✅ Success toast shown, but cart count stays at 0
Status	Open

> Silent success with incorrect data is **harder to detect** than a crash and more confusing for end users — the cart appears to have accepted the item.

5. Selenium WebDriver Automation

Tool: Selenium 4.18.1 + JUnit 5 · Java

File: `automation/selenium/src/test/java/DemoWebShopTest.java`

Test	TC	Assertion	Result
`TC001_shouldLoginWithValidCredentials`	TC-001	Email link visible in header	✅ PASS
`TC002_shouldDisplayResultsForLaptop`	TC-002	`.page-title` + `product-card` > 0	✅ PASS
`TC003_shouldAddProductToCart`	TC-003	Toast visible + cart count ↑	✅ PASS

```
Tests run: 3, Failures: 0, Errors: 0 — BUILD SUCCESS
Total time: 37.205 s (2026-07-06 13:39)
```

Key choices: Selenium Manager (no ChromeDriver setup) · `WebDriverWait` (no sleep) · `localStorage.clear()` for isolation

6. Robot Framework Automation

Tool: Robot Framework 7.1.1 + SeleniumLibrary 6.7.1 · Python
Files: `automation/robot/tests/` · `automation/robot/resources/`

Test	TC	Type	Result
TC-001 Valid Login Shows User Email In Header	TC-001	Positive	✅ PASS
TC-004 Invalid Login Shows Error Message	TC-004	Negative	✅ PASS

Custom keywords: `Open Demo Web Shop` · `Navigate To Login Page` · `Fill Login Form` · `Verify User Is Logged In` · `Verify Login Error Message`

```
TC-001 Valid Login Shows User Email In Header
    Open Demo Web Shop       ${URL}    ${BROWSER}
    Navigate To Login Page   ${URL}
    Fill Login Form          ${VALID_EMAIL}    ${VALID_PASSWORD}
    Verify User Is Logged In ${VALID_EMAIL}
```

7. Postman API Tests

Endpoints under test (`http://localhost:3001`):

Endpoint	Method	Purpose
`/products`	GET	List all products
`/products?name_like=Laptop`	GET	Keyword search
`/cart`	GET / POST	Read / add items
`/users`	GET	List registered users

> Results to be added after Lab 6 (2026-07-09).

Approach: Postman Collection with environment variables (`{{BASE_URL}}`) and automated test scripts validating status codes, response structure, and data integrity.

8. Results Summary

Lab	Tool	Tests	✅ Pass	❌ Fail
Lab 2	JUnit 5	6	6	0
Lab 3	Manual	10	8	2
Lab 4	Selenium	3	3	0
Lab 5	Robot Framework	2	2	0
Lab 6	Postman	TBD	—	—

Total automated: 11 tests · 11 passed · 0 failed
Total defects: 1 open (BUG-001 — qty 0 accepted silently)

9. Lessons Learned

1. **Explicit waits are mandatory for SPAs** — async fetch calls break any fixed sleep
2. **Test isolation requires intent** — `localStorage.clear()` prevented TC-001 ↔ TC-003 state leak
3. **Robot Framework is more readable** — keyword syntax bridges dev/non-dev gap; Selenium gives more control for complex scenarios
4. **Maven Wrapper (`mvnw.cmd`) enables portability** — zero-config Maven for any new machine
5. **Cross-OS node_modules are incompatible** — WSL vs Windows binary mismatch; always install in the target OS
6. **Silent bugs are worse than crashes** — BUG-001 looked like success; only boundary testing revealed it

Conclusion

The Demo Web Shop project exercised the **full testing lifecycle**:

manual design → unit testing → UI automation → API testing

8/10 manual tests pass · 1 confirmed defect · 11 automated tests green

Each tool has its place: JUnit for logic, Selenium for Java-team automation, Robot Framework for readable acceptance tests, Postman for API contracts.

Generated with Claude Code · Team Crousty · TSD 2026